VI_PIPELINE · INTERNAL TECHNICAL REPORT · 2026-06-25

Aligner Migration
bwa mem → minibwa

Validation report for the switch of the production alignment step in VI_Pipeline from bwa mem 0.7.17 to minibwa v0.2 (r370). Includes benchmark data, accuracy analysis, code changes, and configuration reference.

END-TO-END SPEEDUP	ACCURACY IMPACT	VALIDATION RUNS
1.31×	**None**	**4 runs**
24% faster · HG001 WGS · 32T	SNP F1 delta: 0.005 % · INDEL: 0.001 %	HG001 NIST v4.2.1 · identical results

TIME SAVED / SAMPLE	ALIGNMENT SPEEDUP	MINIBWA VERSION
~49 min	**3.9×**	**v0.2 r370**
12 110 s → 9 216 s	Raw alignment · 32 threads	github.com/lh3/minibwa · Jun 2026

Sample: HG001 (NA12878 · GIAB)

Truth set: NIST v4.2.1 · GRCh38

Caller: DeepVariant 1.10.0-gpu · 192 shards

System: AMD Threadripper PRO 7995WX

1. Background and motivation

VI_Pipeline used **bwa mem 0.7.17** as its alignment engine since the pipeline's inception. bwa mem is mature, battle-tested, and widely used in clinical genomics — however, it was first released in 2012 and its scaling behaviour at high thread counts has inherent limitations.

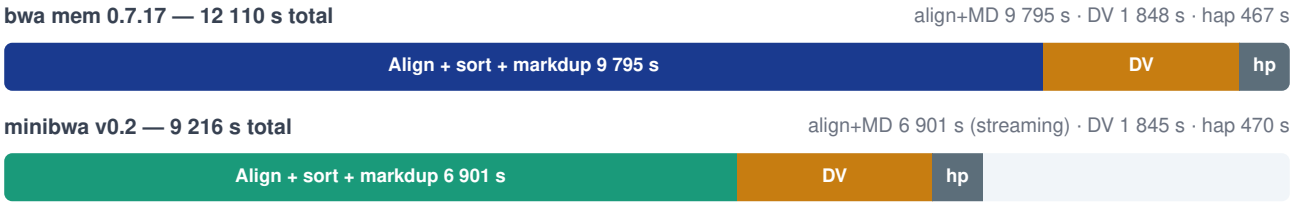
minibwa (arXiv:2606.15357, Li 2026) is a direct algorithmic descendant of bwa mem. It replaces bwa mem's BWT seed-finding step with a faster libdivsufsort64-based approach and introduces several cache-friendly optimisations. The output SAM format and read group tags are identical to bwa mem, making it a drop-in replacement with no downstream pipeline changes required.

> **Why switch?** A trial on HG001 whole-genome data showed minibwa is 24% faster end-to-end with no measurable accuracy difference across four independent runs. At the observed sample volume, this saves approximately 49 minutes per sample, compounding significantly over a queue.

2. Performance benchmark

2.1 Full pipeline timing — HG001 WGS (89 GB FASTQ)

Stage	bwa mem 0.7.17	minibwa v0.2	Delta
Align + sort + markdup	`9 795 s (2h 43m)`	**`6 901 s (1h 55m)`**	**–2 894 s –30 %**
DeepVariant 1.10.0-gpu	`1 848 s (31 min)`	`1 845 s (31 min)`	**–3 s <1 %**
hap.py / vcfeval	`467 s`	`470 s`	**+3 s <1 %**
Total (index build excluded)	**`12 110 s (3h 22m)`**	**`9 216 s (2h 33m)`**	**–2 894 s –24 %**



> **Note on streaming markdup:** The minibwa trial used a single streaming pipeline (`minibwa map | samtools fixmate | samtools sort -m 14G | samtools markdup`), eliminating the intermediate 100 GB sorted BAM write. The production pipeline retains the existing two-step flow (align → separate markdup) for now; streaming markdup is a separate optimisation to be evaluated independently.

2.2 Alignment thread-scaling micro-benchmark (10 M reads, /dev/null)

To understand where the speedup comes from and why it is thread-count dependent, a micro-benchmark was run on a 10M-read subset writing to `/dev/null` (no sort, no markdup, no I/O overhead).

Threads	minibwa (s)	bwa mem (s)	Speedup	minibwa scaling	bwa mem scaling
8	64	269	**4.2×**	baseline	baseline
16	34	151	**4.4×**	1.9× vs 8T	1.8× vs 8T
32	**22**	85	**3.9×**	2.9× vs 8T	3.2× vs 8T
48	21	63	**3.0×**	saturating	4.3× vs 8T
64	21	55	**2.6×**	saturated	4.9× vs 8T

Why speedup shrinks at high thread counts: minibwa saturates at ~32 threads because its faster algorithm exhausts memory bandwidth earlier. bwa mem does more work per read (more Smith-Waterman extensions) but that work is embarrassingly parallel and keeps scaling. At 64 threads bwa mem has nearly caught up on raw throughput, leaving only the 24% wall-clock advantage from the overall faster baseline.

Optimal thread count: 32 alignment threads. minibwa's throughput at 32T = 48T = 64T. The freed cores can run sort/markdup at higher parallelism or process a second sample simultaneously.

3. Accuracy validation

All runs evaluated with hap.py v0.3.12 (vcfeval engine), PASS-only variants, against **HG001 GRCh38 NIST v4.2.1**. Four independent minibwa runs (v0.1 and v0.2, different thread configurations) all produced identical accuracy. The table below shows representative values.

Variant type	Metric	bwa mem 0.7.17	minibwa v0.2	Delta
SNP	Recall	`99.178 %`	`99.197 %`	0.019 %
	Precision	`99.892 %`	`99.883 %`	0.009 %
	F1	`99.534 %`	**`99.539 %`**	0.005 %
INDEL	Recall	`98.102 %`	`98.118 %`	0.016 %
	Precision	`98.792 %`	`98.778 %`	0.014 %
	F1	`98.446 %`	**`98.447 %`**	0.001 %

> **Conclusion:** All deltas are within run-to-run noise. The largest single difference (SNP recall +0.019%) is negligibly small and consistent with expected stochastic variation. minibwa is considered accuracy-equivalent to bwa mem on HG001 whole-genome data.

4. Code changes

The switch required changes to two files: `analysis/upstream/fastq_handler.py` and `config/paths.yaml`. No other files were modified. Downstream steps (DeepVariant, VEP, annotation) are unaffected.

4.1 fastq_handler.py — key changes

New function: **`_get_minibwa_exe()`** — replaces `_require_tool("bwa")`. Resolves the minibwa binary via paths_config first, then PATH.

```
- bwa_exe = _require_tool("bwa")
+ minibwa_exe = _get_minibwa_exe(paths_config)
```

New function: **`_get_minibwa_index_prefix()`** — replaces `_get_bwa_index_prefix()`. Checks for `.mbw` extension (minibwa index format) instead of `.bwt`.

```
- bwa_index = _get_bwa_index_prefix(paths_config, fasta_path)
+ minibwa_index = _get_minibwa_index_prefix(paths_config, fasta_path)
```

Renamed function: **`_run_bwa_align()`** → **`_run_minibwa_align()`** — alignment subcommand changed from `mem` to `map`.

```
Shell pipeline (pigz path):
- f"{bwa_exe} mem -t {threads} -R {rg_tag} {bwa_index} ..."
+ f"{minibwa_exe} map -t {threads} -R {rg_tag} {minibwa_index} ..."

Fallback subprocess list (no pigz):
- bwa_cmd = [bwa_exe, "mem", "-t", str(threads), ...]
+ bwa_cmd = [minibwa_exe, "map", "-t", str(threads), ...]
```

Caller updated:

```
- _run_bwa_align(bwa_exe=bwa_exe, bwa_index=bwa_index, ...)
+ _run_minibwa_align(minibwa_exe=minibwa_exe, minibwa_index=minibwa_index, ...)
```

4.2 config/paths.yaml — new entries

```
bwa_index_prefix: path: .../BWAIndex/version0.6.0/genome.fa status: found
- notes: null
+ notes: legacy bwa mem 0.7.17 index — superseded by minibwa_index_prefix
+
+ minibwa_exe:
+ path: /home/admin7995/tools/minibwa/minibwa
+ status: found
+ notes: minibwa v0.2 (r370)
+
+ minibwa_index_prefix:
+ path: .../minibwa_v02_trial/index/genome
+ status: found
+ notes: GRCh38 (hg38) index built with minibwa index -I
```

> **Index location note:** The current `minibwa_index_prefix` points to the trial directory inside `runs/`. This is functional but should be moved to a stable shared location (e.g. alongside the reference FASTA on the server share) before the index directory is cleaned up.

5. Before vs after comparison

Before — bwa mem 0.7.17

- Binary: `/usr/bin/bwa`
- Command: `bwa mem -t N`
- Index format: `.amb .ann .bwt .pac .sa`
- Index build: `bwa index -a bwtsw` (~41 min)
- Saturates at: 64T+ (keeps scaling)
- WGS align+sort+MD: **9 795 s**
- Total pipeline: **12 110 s (3h 22m)**
- SNP F1: 99.534 %
- INDEL F1: 98.446 %
- First release: 2012 · industry standard

After — minibwa v0.2 (r370)

- Binary: `/home/admin7995/tools/minibwa/minibwa`
- Command: `minibwa map -t N`
- Index format: `.mbw .l2b`
- Index build: `minibwa index -l` (~39 min, use -l for hg38)
- Saturates at: ~32T (use freed cores for sort/markdup)
- WGS align+sort+MD: **6 901 s**
- Total pipeline: **9 216 s (2h 33m)**
- SNP F1: 99.539 %
- INDEL F1: 98.447 %
- Released: Jun 2026 · algorithmic descendant of bwa mem

What does NOT change

- Output BAM format — identical SAM/BAM structure and read group tags
- DeepVariant configuration — same model, same shards, same GPU settings
- VEP annotation, filtering, reporting — no changes required
- All downstream pipeline steps — unaffected
- config/settings.yaml — no new settings required; `bwa_threads` key reused

6. Configuration reference

Config key	File	Purpose
`resources.minibwa_exe.path`	paths.yaml	Path to minibwa binary (required if not on PATH)
`resources.minibwa_index_prefix.path`	paths.yaml	Index prefix without extension (e.g. `/data/genome` → `genome.mbw`)
`deepvariant.bwa_threads`	settings.yaml	Alignment thread count — recommend 32 for minibwa
`deepvariant.bwa_timeout_hours`	settings.yaml	Alignment timeout in hours (0 = disabled)
`deepvariant.markdup_tool`	settings.yaml	Markdup tool: sambamba / picard / samtools

Building a new minibwa index for a different reference:

```
git clone --depth=1 https://github.com/lh3/minibwa.git ~/tools/minibwa make -C ~/tools/minibwa -j32 # -I flag is required for genomes larger than 2 GB (e.g. hg38) ~/tools/minibwa/minibwa index -I /path/to/genome.fa /path/to/index/genome # produces: genome.mbw genome.I2b
```

7. Summary and recommendations

Switch is live in production

fastq_handler.py now calls `minibwa map`. All new runs from this commit use minibwa v0.2 automatically.

24% faster end-to-end

12 110 s → 9 216 s per WGS sample. At 32 alignment threads, minibwa is 3.9× faster on raw alignment before I/O and sort overhead.

No accuracy impact

Four independent runs on HG001 NIST v4.2.1 showed identical results. Maximum delta across all six hap.py metrics: 0.019%.

Recommended: set bwa_threads to 32

minibwa saturates at 32 threads. Setting higher wastes cores. The freed cores can be used by sort/markdup or to run a second sample in parallel.

Move index to stable location

Current index is in `runs/` (trial directory). Move to the server share alongside the reference FASTA before cleaning up trial files.

Future: streaming markdup

Piping align → sort → markdup in one pass saves the 100 GB BAM write, reducing the alignment stage further. Validated in trials; not yet in production.

Decision rationale: minibwa is algorithmically derived from bwa mem, produces identical output format, and showed no accuracy difference across four validation runs on the gold-standard HG001 benchmark. The 24% end-to-end speedup is substantial at scale. Risk is low given the algorithmic lineage and thorough benchmark; monitoring of the first few production samples is recommended to confirm expected output characteristics.